

CV Creation using LLMs

Abstract

This capstone implements an automated CV creation pipeline using two local open-source large language models. Ten unstructured candidate profiles are supplied as text inputs. Gemma 3 1B, hosted locally through Ollama, extracts candidate information into structured JSON and generates concise ATS-friendly CV content. Llama 3.2 1B independently reviews each generated CV for keyword coverage, factual consistency and formatting quality. The application is implemented in Python with a single executable main.py, local model integration, JSON processing and DOCX generation. The design separates extraction, generation and evaluation so that the second model can act as an independent reviewer. Ten sample profiles, ten final CVs, structured outputs and evaluation files are included. Deterministic fallbacks allow inspection when Ollama is unavailable, while the intended graded workflow uses both LLMs.

1. Introduction

CV creation is repetitive because candidate information often arrives as unstructured text and must be reorganized into a professional, ATS-friendly format. The project demonstrates how local LLMs can automate this transformation while keeping candidate data on the local machine. The topic is motivated by the need for faster resume preparation, consistent formatting and better ATS-oriented keyword coverage.

2. Problem Statement

Given unstructured user profile information, create a structured representation and a ready-to-use CV without inventing candidate facts. The system must use at least two LLMs and provide measurable review information for generated CVs.

3. Objectives

- Extract name, contact, education, experience, skills, achievements and projects.
- Generate concise ATS-friendly CVs.
- Use a second LLM for independent evaluation.
- Produce ten final CVs and supporting outputs.
- Support local/private execution using Ollama.

4. Methodology

Workflow: unstructured profile → Gemma extraction → structured JSON → Gemma CV generation → Llama review → DOCX CV and evaluation JSON. Gemma 3 1B was selected because the supplied project outline identifies it as a lightweight open-source model suitable for local, privacy-centric resume tasks. Llama 3.2 1B is used as a separate reviewer so generation and evaluation are not dependent on the same model. Python, requests and python-docx provide orchestration and document generation.

5. System Design / Implementation

The implementation is intentionally flat to comply with the capstone submission rules. main.py contains model calls, JSON extraction, fallback extraction, CV generation, review and DOCX creation. Each profile is processed independently. Model names and the Ollama host are command-line parameters; no API key is embedded in source code.

6. Results and Analysis

The submission contains 10 unstructured profile inputs and 10 final CV documents, plus 10 structured JSON files, 10 reviewer JSON files and evaluation_summary.json. Evaluation tracks overall score, keyword coverage, factual consistency and format score. The included sample outputs use deterministic fallback so the package is inspectable before Ollama execution; the intended final run is with Gemma 3 1B and Llama 3.2 1B.

7. Conclusion and Future Work

The project demonstrates an end-to-end local LLM pipeline for CV creation with a second-model evaluation stage. Future extensions can add job-description parsing, job-specific CV tailoring, user editing and iterative refinement, ATS keyword feedback, PDF generation and a web UI.

8. References

1. HAAI++ Capstone Project Instructions, HPPCS01: CV Creation using LLMs.
2. CV Creation using LLMs project outline: model selection, extraction, generation, review and evaluation.
3. Supplied Project Report template.

Acknowledgement

I acknowledge the HAAI++ capstone guidance and supplied project materials used to define the scope, submission structure and evaluation approach.